

◆ 1. Tokenization

Definition	Breaking a raw text sentence into individual words (tokens)
File + Function	<code>preprocess.py</code> → <code>tokenize(text)</code>
Example from code	<code>word_tokenize("where is my order")</code> → <code>["where", "is", "my", "order"]</code>
Why needed	Computers can't process full sentences; they need word-level units

◆ 2. Text Preprocessing

Definition	Cleaning raw text to remove noise before NLP processing
File + Function	<code>preprocess.py</code> → <code>preprocess(text)</code> (master pipeline)
Example from code	<code>"Where IS my Order?!"</code> → lowercased → punctuation removed → tokenized → stopwords removed → <code>["order"]</code>
Why needed	"ORDER", "order", "Order!" should all mean the same thing to our model

◆ 3. Stopword Removal

Definition	Removing common words (is, the, my, a) that don't carry meaningful information
File + Function	<code>preprocess.py</code> → <code>remove_stopwords(tokens)</code>
Example from code	<code>["where", "is", "my", "order"]</code> → <code>["order"]</code> (is/my/where are stopwords)
Why needed	These words dilute TF-IDF scores; removing them improves matching accuracy

◆ 4. Intent Classification (TF-IDF)

Definition	Identifying the <i>purpose</i> or <i>goal</i> behind user's message
File + Function	<code>intent_classifier.py</code> → <code>classify(user_input)</code>
Example from code	User: <i>"where is my order"</i> → TF-IDF + cosine similarity → <code>order_status</code> intent (score: 0.87)
Why TF-IDF?	TF-IDF gives higher weight to rare/important words and lower weight to common ones — perfect for short queries

How TF-IDF works (for viva):

- **TF** = how often a word appears in one document

- **IDF** = penalizes words that appear in many documents (common words)
- **TF × IDF** = important, rare words get high scores
- **Cosine Similarity** = measures angle between two vectors; 1.0 = identical, 0.0 = completely different

◆ 5. Named Entity Recognition (NER)

Definition	Automatically detecting and classifying real-world entities in text (names, IDs, places, numbers)
File + Function	<code>ner.py</code> → <code>extract_entities(text)</code>
Example from code	"Track order ORD-12345" → {"order_id": "ORD-12345", "product": None, "phone": None}
Method used	Rule-based NER using Regular Expressions (Regex)
Why regex?	Order IDs always follow a fixed pattern <code>ORD-XXXXXX</code> — regex is perfect for structured patterns

◆ 6. Response Generation

Definition	Producing a meaningful reply based on detected intent and entities
File + Function	<code>chatbot.py</code> → <code>personalize_response(response, entities)</code>
Example from code	Template: "Order {order_id} has shipped" + entity <code>ORD-48291</code> → "Order <code>ORD-48291</code> has shipped"
Why template-based?	Simple, reliable, fast — no neural network needed; perfect for FAQ-style support

Q1. What is the main NLP technique used for intent classification in your project?

"I used TF-IDF (Term Frequency-Inverse Document Frequency) vectorization combined with Cosine Similarity. The chatbot converts both the user input and all known patterns into TF-IDF vectors, then finds which pattern is most similar to the user's query using cosine similarity."

Q2. Why did you use TF-IDF instead of simple keyword matching?

"Keyword matching would fail for paraphrased inputs — for example, 'track my package' and 'where is my order' both mean the same thing but share no keywords. TF-IDF captures the statistical importance of words, so it generalizes better. It also handles synonyms indirectly through weighted scoring."

Q3. What is cosine similarity and why is it used here?

"Cosine similarity measures the angle between two vectors in n -dimensional space. A score of 1.0 means the vectors point in the same direction (identical meaning), while 0.0 means perpendicular (no relation). We use it because for text comparison, the direction of the vector (what words appear) matters more than the magnitude (how many words)."

Q4. What is the difference between Rule-based NER and ML-based NER?

"Rule-based NER uses predefined patterns — like regex — to detect entities. It's fast, interpretable, and works perfectly when entities follow a fixed format (like our Order IDs: ORD-XXXXX). ML-based NER (like spaCy's transformer model) learns from labeled data and handles variable entity patterns. For this project, rule-based was simpler and sufficient."

Q5. Why did you remove stopwords? Can removing them cause problems?

"Stopwords like 'is', 'the', 'my' don't help distinguish between intents, so removing them improves TF-IDF accuracy. However, yes — removing stopwords can cause issues in sentiment analysis ('not good' → 'good' after removing 'not') or in queries where stopwords carry meaning. For intent classification in customer support, it works well."

Q6. How does your chatbot handle inputs it doesn't understand?

"I implemented a confidence threshold of 0.15 in the classify() function. If the cosine similarity score of the best match is below this threshold, the chatbot returns a fallback response asking the user to rephrase. This prevents the bot from giving irrelevant answers with false confidence."

Q7. What is tokenization and which library did you use for it?

"Tokenization is the process of splitting a continuous text string into a list of individual word tokens. I used NLTK's word_tokenize() function, which is smarter than simple split() because it correctly handles contractions (it's → it + 's), punctuation attached to words, and other edge cases."

Q8. How many intents does your chatbot support and how did you define them?

"The chatbot supports 15 intents: greeting, goodbye, order status, order ID provided, return request, refund status, product complaint, delivery issue, cancel order, payment issue, product info, discount/coupon, contact agent, warranty, and thank you. All intents are defined in data/intents.py with multiple sample patterns each — the more patterns per intent, the better the TF-IDF model performs."

Q9. What is the difference between a chatbot and a conversational AI? Where does your project fall?

"A rule-based/retrieval-based chatbot (like mine) selects responses from predefined templates based on pattern matching — it doesn't generate new sentences. Conversational AI (like ChatGPT) uses large language models that generate novel responses from scratch. My project is a retrieval-based chatbot with NLP preprocessing —

suitable for FAQ-style customer support where predictable, accurate answers are more important than creative responses."

Q10. How would you improve this chatbot if given more time?

"Several improvements are possible: (1) Add context/conversation history so the bot remembers previous messages in the same session, (2) Use spaCy's pre-trained NER model for more robust entity extraction, (3) Integrate a small SQLite database to actually look up real order data by ID, (4) Add sentiment analysis to detect frustrated users and auto-escalate to human agents, (5) Use ML classification (SVM or Naive Bayes) instead of pure cosine similarity for better intent accuracy."

Quick Reference Cheat Sheet

text



FILE	NLP CONCEPT	KEY FUNCTION
preprocess.py	Tokenization	tokenize()
preprocess.py	Lowercasing	to_lowercase()
preprocess.py	Punctuation Removal	remove_punctuation()
preprocess.py	Stopword Removal	remove_stopwords()
preprocess.py	Full Pipeline	preprocess()
intent_classifier.py	TF-IDF Vectorization	_build_model()
intent_classifier.py	Cosine Similarity	classify()
intent_classifier.py	Intent Classification	classify()
ner.py	NER - Order IDs	extract_order_id()
ner.py	NER - Products	extract_product_name()
ner.py	NER - Phone Numbers	extract_phone_number()
ner.py	Full NER Pipeline	extract_entities()
chatbot.py	Response Generation	personalize_response()
chatbot.py	CLI Loop	main()
data/intents.py	Knowledge Base	INTENTS dictionary

💡 **Pro Tips for Viva:**

- Always say "**preprocess.py handles the NLP pipeline**" — it shows you understand separation of concerns
- Mention the **threshold value (0.15)** in `classify()` — shows you thought about edge cases
- If asked "why not deep learning?" → "Classical NLP is interpretable, fast, requires no GPU, and is perfect for small-domain chatbots with well-defined intents"
- Enable `SHOW_DEBUG = True` in `chatbot.py` during demo to show intent scores — **examiners love seeing live confidence scores**

💡 **Viva Tip:** If your examiner asks "How did you manage dependencies?" — say:
"I used Python's built-in `venv` module to create an isolated virtual environment, installed all dependencies inside it, and saved them to `requirements.txt` using `pip freeze`. This ensures the project runs consistently on any machine."

That's a **very impressive answer** for a college viva! 🎯

Bonus Viva Point:

If asked about logging — say:

"I added a `logger.py` module that creates a timestamped log file for every session inside a `logs/` directory. This simulates real customer support ticketing systems where every chat is recorded."

That's a **great extra feature** to mention! 🎯

💡 **Viva Tip:** This log analysis is actually a gold mine for your viva!

You can say:

"I tested the chatbot and identified failure cases — like single-word inputs matching greedily and negative phrases matching wrong intents. I then fixed these by raising the confidence threshold, adding a single-word guard, and introducing a frustration intent."

That shows **testing + iteration mindset** — exactly what examiners love to see! 🎯